Downrange - Analysis

Schritt 1: Kräfte auf die Rakete

Es wirken drei Kräfte

- F_{Schub} nach oben
- $F_g = mg$ nach unten
- $F_{Luft} = \frac{1}{2} \rho c_w A v^2$ gegen die Bewegungsrichtung

Schritt 2: Masseänderung

Die Rakete wird leichter, weil Treibstoff verbrennt

$$m(t) = m_0 - \dot{m} t,$$

wobei m_0 die Startmasse und $\dot{m}$ der Massestrom ($kg \cdot s^{-1}$) ist.

$$\dot{m} = - \frac{m_{propellant}}{t_{burn}}$$

Schritt 3: Zerlegung in x und y

Der Schubvektor zeigt unter dem Winkel θ zur Horizontalen

x-Richtung: $F_x = m(t) \cdot \ddot{x} = F_{Schub} \cdot \cos\theta - F_{Luft} + F_{Wind}$

y-Richtung: $F_y = m(t) \cdot \ddot{y} = F_{Schub} \cdot \sin\theta - m(t) g - F_{Luft}$

Schritt 4: Relativgeschwindigkeit und Luftwiderstand

Die Rakete bewegt sich relativ zum Wind

$$v_{rel} = v_{Rakete} - v_{Wind}$$

$$\vec{F}_{Luft} = \frac{1}{2} \rho c_w A |\vec{v}_{rel}| \cdot \vec{v}_{rel} \quad \text{mit} \quad |\vec{v}_{rel}| = \sqrt{v_{rel,x}^2 + v_{rel,y}^2}$$

Schritt 5: Zustandsvektor (State Space)

Vier Variablen, vier gekoppelte gewöhnliche DGL erster Ordnung

$$F_x = m a_x = m \ddot{x} \iff \ddot{x} = \frac{F}{m}, \quad \dot{x} := v_x$$

$$\Rightarrow \dot{v}_x = \frac{1}{m(t)} \left(F_{Schub} \cdot \cos\theta - F_{Luft,x} + F_{Wind} \right)$$

$$F_y = m a_y = m \ddot{y} \iff \ddot{y} = \frac{F}{m}, \quad \dot{y} = v_y$$

$$\Rightarrow \dot{v}_y = \frac{1}{m(t)} \left(F_{Schub} \cdot \sin\theta - m(t) \cdot g - F_{Luft,y} \right)$$

$$\Rightarrow \quad Y = (x, v_x, y, v_y)$$

$$\dot{Y} = (v_x, \ddot{x}, v_y, \ddot{y}),$$

wobei $\ddot{x}$ und $\ddot{y}$ aus Schritt 3 stammen.

Schritt 6: Zwei Stufen

Erste Stufe brennt von $t = 0$ bis $t = t_{b1}$. Bei t_{b1} wird die erste Stufe abgetrennt, plötzlicher Massenverlust.
Dann zündet die zweite Stufe mit neuen Parametern (anderer Schub, andere Masse, andere Brennzeit).
Nach t_{b2} fliegt die Rakete ballistisch weiter

Abschnitt 1 ($t = 0$ bis t_{b1}): Bei t_{b1} den Zustandsvektor (x, vx, y, vy) für Anfangsbedingungen
für Abschnitt 2 und abgetrennter erster Stufe festhalten
und neue Schubparameter festlegen.
$\Rightarrow$ Neues DGL

Abschnitt 2 (t_{b1} bis $t_{b1} + t_{b2}$): DGLs mit neuen Parametern wie in Abschnitt 1 erneut numerisch berechnen

Abschnitt 3 (nach $t_{b1} + t_{b2}$): kein Schub, ballistischer Flug

Schritt 7: Monte Carlo

Jeder Parameter hat eine Unsicherheit

- F_{schub} mit $N(\mu, \sigma)$: $100\,kN \pm 3\%$
- v_{Wind} mit $N(\mu, \sigma)$: $5\,\frac{m}{s} \pm 2\,\frac{m}{s}$
- m_0 mit $N(\mu, \sigma)$: $500\,kg \pm 1\%$
- θ mit $N(\mu, \sigma)$: $85° \pm 0{,}5°$

Konfidenzintervall von 95: ci_95 = np.percentil („x-Koordinate", [2.5, 97.5])

Code Struktur

rocket_simulator.py

```
class StageParameters
    ⋮
    def validate(self) -> None:
    ⋮
    @property (als Variable wiederverwendbar)
    def m_dry(self) -> float:
class RocketSimulator
    def __init__(...)
    ⋮
    def _rocket_eom(...) -> List   (private function just for equation of motion that returns the state vector)
    ⋮
    return [vx, ix, vy, iy]
    def _ballistic_eom(...) -> List
    ⋮
    @staticmethod
    def _hit_ground(state)
    def simulate(...) -> „2x Tuple of floats & Dict":
        ⋮
        solve_ivp(...)   (for all stages)
        ⋮
        return payload_impact, stage1_impact, trajectory
    def monte_carlo(...) -> „Tuple of two arrays"
        return payload_impacts, stage1_impacts
```

config.json

- Simulation
- environment
- stage1
- stage2
- monte_carlo

main.py

```
from ... import StageParameters, RocketSimulator
def load_config
    ⋮
def load_single_trajectory(...):
    ⋮
    x_all = np.concatenate([„stage1_burn"], [„stage2_burn"], [„ballistic"]
    ⋮
def plot_monte_carlo(...):
    „3 Subplots"
if __name__ = "__main__":
```

1. Load config
2. Build StageParameters
3. Create Simulation
4. Plot normal trajectory
5. Run Monte Carlo
6. Plot Results